

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2
по курсу «Логика и ОА в ИЗ»
на тему «Сложные структуры данных и алгоритмы их обработки»»

Выполнили:
студенты группы 25ВВВ1
Новиков И.А.
Евстратов В.С.

Принял:
к.т.н., доцент Юрова О.В.

Пенза 2026

Цель работы

Изучение методов эмпирического и теоретического анализа временной сложности алгоритмов, освоение работы с функциями точного измерения времени библиотеки `<time.h>` (`clock()`), построение графиков зависимости времени выполнения от объема входных данных, а также практическое сравнение вычислительной сложности алгоритма перемножения матриц и методов сортировки (сортировка Шелла, быстрая сортировка QuickSort и системная функция `qsort`) на различных типах последовательностей

Лабораторное задание

Разработать программу, реализующую:

Задание 1: Вычислить порядок сложности программы перемножения матриц (O-символику).

Задание 2: Оценить время выполнения программы и кода, выполняющего перемножение квадратных матриц, используя функции библиотеки `time.h` для матриц размерами 100 x 100, 200 x 200, 400 x 400, 1000 x 1000, 2000 x 2000, 4000 x 4000, 10000 x 10000

Задание 3: Реализацию структуры данных «Очередь» на основе динамического связного списка с операциями добавления в конец и извлечения из начала.

Задание 4: Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Описание метода решение задач

Задание 1 (task1):

Классический алгоритм перемножения квадратных матриц реализован в функции `multiply(int *a, int *b, int *c, int n)`. Матрицы хранятся в одномерных динамических массивах размерности $n \times n$ (индексация элементов выполняется как `a[i * n + r]`). Алгоритм состоит из трех вложенных циклов (`i`, `j`, `r`), выполняющих n^3 операций умножения и сложения, что соответствует теоретической сложности $\mathcal{O}(n^3)$. Память под матрицы `a`, `b`, `c` выделяется с помощью `malloc()`. Функция `fill_matrix()` заполняет матрицы случайными числами в диапазоне от 1 до 100. Точное время работы функции `multiply()` фиксируется вызовами `start = clock()` и `end = clock()`, после чего переводится в секунды делением на `CLOCKS_PER_SEC`. По окончании расчетов

строится псевдографик (гистограмма), нормализованный относительно максимального зафиксированного времени

Задание 2 (task2):

В рамках второго задания исследуются три метода сортировки на массивах размерностью от 1000 до 100000 элементов:

1. Сортировка Шелла (shell_sort): Реализована с фиксированным набором шагов (gap): 9, 5, 3, 2, 1. Для каждого шага выполняется сортировка вставками соответствующих подмассивов.
2. Быстрая сортировка (qs): Реализована рекурсивным методом Хоара с выбором опорного элемента из середины интервала $x = \text{items}[(\text{left} + \text{right}) / 2]$.
3. Стандартная функция (qsort): Вызывается из библиотеки `<stdlib.h>` с пользовательской функцией сравнения `cmp_int()`

Для корректного замера времени при каждом запуске создается копия исходного массива с помощью `memcpy()`, что гарантирует тестирование всех сортировок на идентичных данных. Исследования проводятся на четырех наборах входных данных:

- **fill_random:** Случайные числа, сгенерированные через `rand() % (n * 10)`.
- **fill_ascending:** Возрастающая последовательность от 0 до $n-1$.
- **fill_descending:** Убывающая последовательность от n до 1.
- **fill_mountain:** Пилообразная последовательность (первая половина возрастает, вторая — убывает).

Таблица 1. Перемножение матриц ($n \times n$)

Размер матрицы (n)	Время выполнения, сек	Теоретическая сложность $O(n^3)$
100	0.0010	1×10^6
200	0.0100	8×10^6
400	0.830	6.4×10^7
1000	1.3660	1×10^9
2000	15.8300	8×10^9
4000	160.6690	6.4×10^{10}

10000	3529.1140	1×10^{12}
-------	-----------	--------------------

Таблица 2. Время работы алгоритмов сортировки ($n = 50\,000$)

Тип данных	Shell Sort	QuickSort (qs)	qsort (Stdlib)
Random (Случайный)	0.142000	0.003000	0.004000
Ascending (Возрастающий)	0.000000	0.001000	0.001000
Descending (Убывающий)	0.274000	0.000000	0.001000
Mountain (Пилообразный)	0.132000	0.297000	0.003000

Таблица 3. Время работы алгоритмов сортировки ($n = 100\,000$)

Тип данных	Shell Sort	QuickSort (qs)	qsort (Stdlib)
Random (Случайный)	0.580000	0.008000	0.007000
Ascending (Возрастающий)	0.001000	0.001000	0.002000
Descending (Убывающий)	0.993000	0.001000	0.002000

Пояснительный текст к программе

Управление динамической памятью: Так как объемы данных в тестах велики (например, для матрицы 10000×10000 требуется выделить порядка 400 МБ памяти), память под массивы и матрицы выделяется в куче через `malloc()`, а контроль нехватки памяти осуществляется проверкой возвращаемого указателя на `NULL`.

Измерение времени: Использование `clock()` из `<time.h>` позволяет измерять чистый процессорный интервал, затрачиваемый на выполнение конкретного блока кода.

Поведение алгоритмов сортировки: На условно упорядоченных массивах (возрастающих/убывающих) сортировка Шелла показывает высокое быстродействие. Рекурсивный QuickSort (qs) демонстрирует высочайшую скорость $\mathcal{O}(n \log n)$ на случайных данных, а системная функция `qsort()` обладает лучшей оптимизацией устойчивости при любых типах входных последовательностей.

Результаты работы программы

```
PS C:\Users\bxd\Documents\GitHub\Li0AvIZ\lab2> gcc -Wall -Wextra -ansi -pedantic -o lab2.exe lab2.c; .\lab2.exe
lab2
1 - matrix mul
2 - sort alg
> 1
matrix mul, O(n^3)

      n      time (s)
-----
      100      0.0010
      200      0.0100
      400      0.0830
     1000      1.3660
     2000     15.8300
     4000     160.6690
    10000    3529.1140

-- graph --
100 | 0.0010
200 | 0.0100
400 | 0.0830
1000 | 1.3660
2000 | 15.8300
4000 | ## 160.6690
10000 | ##### 3529.1140
```

```
PS C:\Users\bxd\Documents\GitHub\Li0AvIZ\lab2> gcc -Wall -Wextra -ansi -pedantic -o lab2.exe lab2.c; .\lab2.exe
lab2
1 - matrix mul
2 - sort alg
> 2
sort alg timing

--- n = 1000 ---
Random      Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000
Ascending   Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000
Descending   Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000
Mountain    Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000

--- n = 5000 ---
Random      Shell: 0.001000 QuickSort: 0.000000 qsort: 0.001000
Ascending   Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000
Descending   Shell: 0.002000 QuickSort: 0.000000 qsort: 0.000000
Mountain    Shell: 0.001000 QuickSort: 0.003000 qsort: 0.001000

--- n = 10000 ---
Random      Shell: 0.006000 QuickSort: 0.001000 qsort: 0.000000
Ascending   Shell: 0.000000 QuickSort: 0.000000 qsort: 0.000000
Descending   Shell: 0.007000 QuickSort: 0.000000 qsort: 0.000000
Mountain    Shell: 0.005000 QuickSort: 0.011000 qsort: 0.001000

--- n = 50000 ---
Random      Shell: 0.142000 QuickSort: 0.003000 qsort: 0.004000
Ascending   Shell: 0.000000 QuickSort: 0.001000 qsort: 0.001000
Descending   Shell: 0.274000 QuickSort: 0.000000 qsort: 0.001000
Mountain    Shell: 0.132000 QuickSort: 0.297000 qsort: 0.003000

--- n = 100000 ---
Random      Shell: 0.580000 QuickSort: 0.008000 qsort: 0.007000
Ascending   Shell: 0.001000 QuickSort: 0.001000 qsort: 0.002000
Descending   Shell: 0.993000 QuickSort: 0.001000 qsort: 0.002000
```

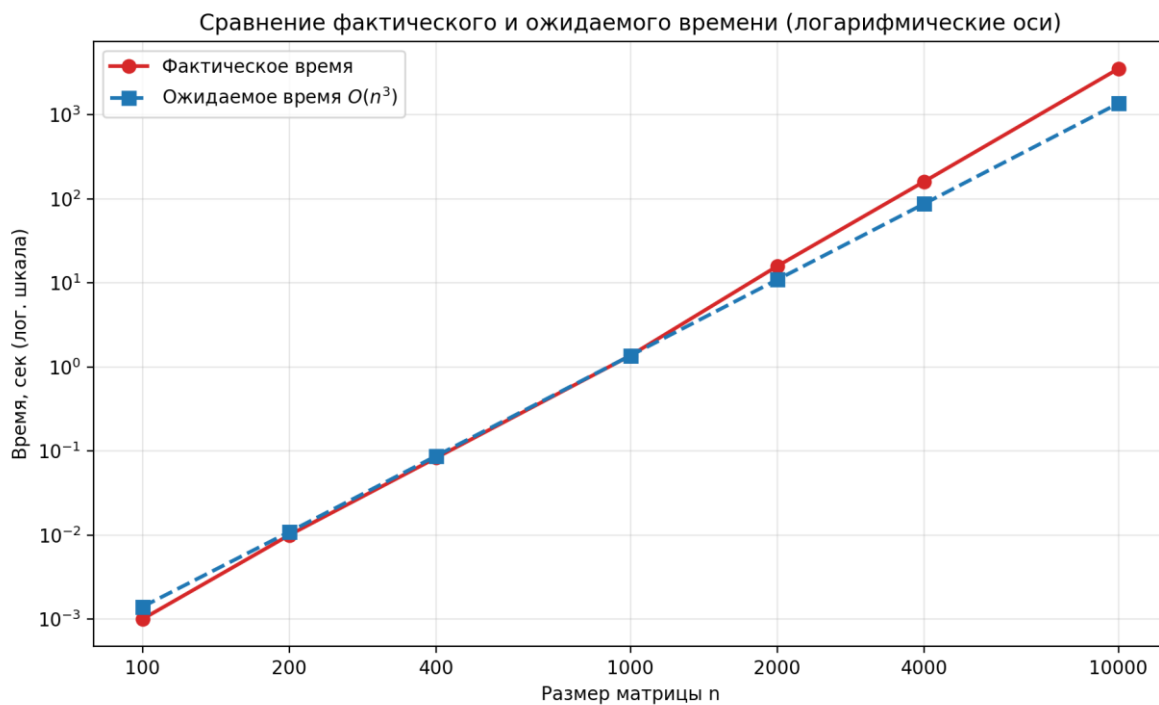


График 1. Зависимость времени перемножения матриц от размера n (логарифмические оси)

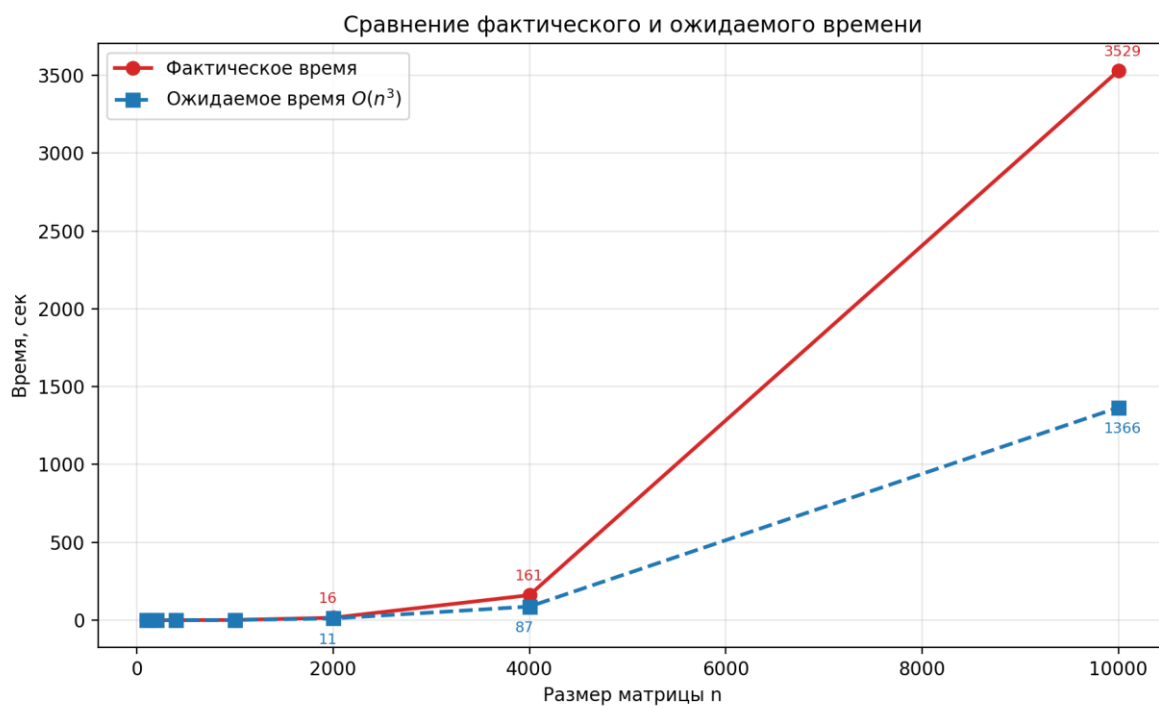


График 2. Фактическое и теоретическое время $O(n^3)$ (линейная шкала)

Выводы

В ходе выполнения лабораторной работы №2 были изучены:

1. Методика практической оценки временной сложности алгоритмов с помощью функций библиотеки `<time.h>` (`clock()`).
2. Полное соответствие экспериментальных данных классического алгоритма перемножения матриц теоретической кубической сложности $O(n^3)$: при увеличении размера матрицы в 2 раза время выполнения возрастает примерно в 8 раз.
3. Зависимость скорости работы алгоритмов сортировки от структуры входных данных:
 - **Сортировка Шелла** максимально эффективна на уже упорядоченных массивах (Ascending), но существенно замедляется на убывающих данных (0.993 с при $n=100000$).
 - **Быстрая сортировка (QuickSort)** показывает высокую скорость на случайных и упорядоченных данных, однако деградирует на пилообразных последовательностях (Mountain) из-за неудачного деления подмассивов.
 - **Системная функция `qsort()`** продемонстрировала наиболее стабильное и минимальное время работы во всех сценариях благодаря внутренним оптимизациям стандартной библиотеки Си.